

Enabling Data-Deduplication-Assisted Data Relocation for Interlaced Magnetic Recording

Chen-Jui Tu

National Yang Ming Chiao Tung University
Taiwan

Email: raytu.tpe@gmail.com

Shuo-Han Chen*

National Yang Ming Chiao Tung University
Taiwan

Email: shch@nycu.edu.tw

Abstract—Interlaced Magnetic Recording (IMR) drives have been regarded as promising hard disk drives (HDDs) to meet the ever-growing storage demand in the post-AI era. Among various technologies, IMR drives achieve increased storage capacity by overlapping tracks in an interlaced fashion; however, updating data on overlapped tracks necessitates rewriting up to two tracks, which can degrade performance. Previous strategies have attempted to reduce track rewrites by either delaying the timing of overlapping based on storage space usage or relocating frequently updated data to non-overlapped tracks through update-frequency-based hot/cold data separation. In contrast, this paper proposes a novel approach that leverages data deduplication as a natural solution to the rewrite challenge in IMR drives. Unlike conventional hot/cold separation, this paper utilizes deduplication metadata, specifically reference counts, as a direct indicator for data relocation. The rationale behind the proposed approach is that deduplicated data, which remains static unless reference counts drop to one, can be stored or migrated to overlapped tracks without inducing future track rewrites. The proposed approach diverges from traditional frequency tracking methods by leveraging data deduplication information to guide data placement and reduce rewriting. Evaluation results show that the proposed scheme effectively reduces the accumulated read/write latency by 37.57% on average compared with state-of-the-art data management on IMR drives with data deduplication enabled.

Index Terms—IMR, deduplication, relocation, HDD

I. INTRODUCTION

Owing to the explosive growth of digital data in the post-AI era, both hardware and software designs have been explored to either enhance the storage density or reduce the data volume without significantly changing existing file and storage systems. Among various hardware solutions, hard disk drives (HDDs) are favored for their low cost-per-gigabyte, high reliability, and long lifespan. While the superparamagnetic effect (SPE) [1] limits the storage density of existing conventional magnetic recording (CMR) HDDs, various new technologies, such as the shingled magnetic recording (SMR) [2–4] and interlaced magnetic recording (IMR) [5–7], have been widely studied. Although SMR has been commercialized [8], the shingled track layout leads to an excessive number of data rewrites under random workload and greatly limits the random write performance. Thus, SMR becomes less preferable and issues related to unpredictability of I/O performance have also been reported [3, 4, 9]. Recently, the advances of interlaced magnetic recording (IMR) technology have outperformed SMR and enables even-higher storage density with alleviated track rewrites.

Figure 1 compares CMR, SMR and IMR technologies. Unlike SMR that has multiple tracks overlapped on top of previous tracks, IMR organizes *top tracks* and *bottom tracks* in an

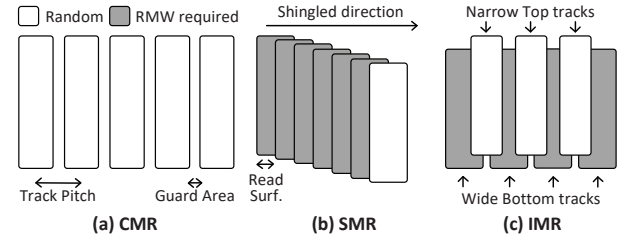


Fig. 1. Comparison of CMR, SMR, and IMR, in which the track pitch and the guard area indicate the distance between data tracks and the distance to avoid interference. For RMW-required tracks, read surfaces are reserved and exposed for the disk read head to retrieve stored data.

interlaced fashion and each top track overlaps at most two adjacent bottom tracks. IMR achieves higher linear density by applying higher laser current to the wide bottom tracks and has narrow top tracks placed closer for higher areal density [10]. Such an interlaced layout enables top tracks to be updated freely and induces at most two track rewrites when updating bottom tracks if adjacent top tracks contain valid data. As a result, IMR is considered a more attractive alternative over SMR with the lowered number of track rewrites. To further alleviate the overhead when updating bottom tracks on IMR drives, space allocation strategies, including the rewrite-minimized data placement [11], the three-phase data management [12], and the Z-Alloc [13], are proposed to postpone the allocation of top track until all bottom tracks are in use. After top tracks are allocated, another perspective is to reduce updates on bottom tracks by either buffering updates on free top tracks, such as the top buffer [13] and selective track buffering [14], or relocating frequently-updated bottom tracks to non-overlapped top tracks. Examples of data relocation include the block swap [13], the track flipping [14], and the data hotness aware movement [15]. Nevertheless, *most existing mechanisms employ similar approaches to swap or relocate data blocks based on update frequency for hot/cold data separation.*

Examples of software solutions for enlarging usable storage space include the data compression and data deduplication techniques [16]. While data compression encodes the digital data using fewer bits, data deduplication eliminates duplicate copies of repeated data. Although both techniques can effectively lower data size, data compression has higher performance overhead since encoding and decoding are required during data writes and reads, respectively. In addition, owing to the decoding overhead, the *readahead*¹ [17] mechanism in existing file systems may induce additional decoding overhead if the access pattern prediction is incorrect. In other words, data deduplication is preferable with read performance consideration.

¹By predicting the access pattern, the readahead mechanism prefetches file data into main memory to reduce the latency of future access.

This work was supported in part by the National Science and Technology Council, Taiwan, under grant nos. 113-2221-E-A49-189-MY3, 111-2221-E-A49-198-MY3, and 113-2221-E-007-134.

*Corresponding author

Data deduplication identifies and eliminates redundant copies of data, retaining a single instance at block- or chunk-level by (1) calculating the *cryptographically secure hash-based fingerprints* [16], (2) comparing fingerprints to identify duplication, (3) storing only one copy of the duplicate data, (4) maintaining mapping information between the stored data and those deduped chunks or files. Even though data deduplication techniques have been studied for over 20 years [16] and machine learning deduplication has also been proposed [18, 19], *the potential of leveraging data deduplication information to facilitate the IMR management received much less attention.*

Notably, instead of utilizing data deduplication information for tracking update frequency and separating hot/cold data, this paper is constructed based on the fact that *deduplicated data chunks² remain unchanged unless the reference count drops to one.* Specifically, deduplicated data chunks do not receive updates directly. Instead, updates are applied to new or unique chunks. If the reference count of a duplicated chunk drops to one, suggesting it is no longer shared, it may then become eligible for modifications or updates. This paper introduces a *deduplication-assisted data relocation (DADR)* scheme to reduce track rewrites by utilizing data deduplication information, specifically the reference count of each block or chunk, enabling effective data relocation and space allocation. Integrating data deduplication onto IMR drives requires sharing deduplication information between the host system (e.g., file systems) and the disk firmware (e.g., Interlaced Translation Layer (ITL) [14]). This is because, in the proposed scheme, fingerprint calculation is assumed to be conducted at the host system, as IMR drives are typically resource-constrained. The deduplication information is then maintained within ITL, based on details passed down from the host system as a *hint* via the ATA command set [20, 21].

The DADR scheme is threefold. First, after valid top tracks start overlapping bottom tracks, non-deduped bottom tracks are swapped with deduped top tracks during bottom track updates to reduce the number of track rewrites. Meanwhile, during track rewrites, deduped top tracks are actively relocated to invalid or non-deduped bottom tracks. Second, after fingerprint calculation is performed on the host system, the proposed DADR scheme actively relocates non-deduped and deduped data writes to top and bottom tracks. Notably, since the size of deduped blocks or chunks (i.e., 4 KB) is typically smaller than a IMR tracks (i.e., 2 MB), non-deduped and deduped data chunks are packed into non-deduped and deduped data tracks, respectively. Third, the DADR scheme delays the allocation of top tracks that are on top of non-deduped bottom tracks. Above measures are effective since deduped data chunks are static as long as the reference count is larger than 1, thus allowing the proposed scheme to effectively lower track rewrites on IMR drives. Another benefit of integrating data deduplication onto IMR drives is lowering the total amount of written data, thus further reducing the overall latency during data writes. The main contribution of this study lies in *leveraging the deduplication information to facilitate the IMR management, reducing track rewrites and enhancing performance.* Evaluation results show that the number of track rewrites and the accumulated latency are reduced by 41.34% and

37.75% on average compared with the state-of-the-art ITL [13] with data deduplication, Z-alloc, block swap, and top buffer enabled.

II. BACKGROUND AND MOTIVATION

A. Interlaced Magnetic Recording (IMR)

Interlaced Magnetic Recording (IMR) technology achieves higher storage density by organizing data tracks in an interlaced layout, as shown in Figure 1(c). By placing narrower top tracks on at most two wider bottom tracks, IMR is capable of reading data from bottom tracks through the residual track surface. Nevertheless, due to the track overlap, writing data on bottom tracks destroys valid data on adjacent top tracks. To restore those destroyed data, the *read-modify-write (RMW)* operation is introduced. As illustrated in Figure 2(a), before *Track 1* is updated, data on *Tracks 8* and *9* are first read out. Then, after updates to *Track 1* are already merged, *Tracks 1, 8* and *9* are then written back. Inevitably, additional write traffic is induced due to track rewrites during the RMW operations and becomes performance overhead. To lower the amount of write traffic and the number of track rewrites, existing strategies can be categorized into 3 groups, including space allocation, track relocation, and track buffering [12–14]. Notably, these strategies are typically implemented in the interlaced translation layer (ITL) to manage IMR drives with a *logical-to-physical (L2P) mapping table*. For instance, based on the ratio of space utilization, the Z-alloc [13] has three phases of allocation, and the allocation direction is reversed when phases switch.

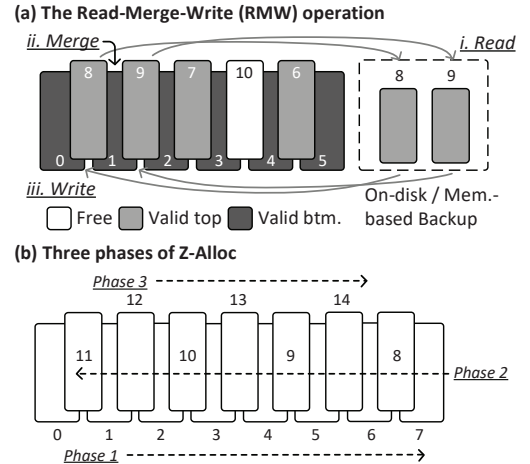


Fig. 2. Read-merge-write operation and Z-alloc of IMR drives, in which track indexes are labeled according to their allocation sequence.

As shown in Figure 2(b), the *Phase 1* of Z-alloc only allocates bottom tracks. Then, the *Phase 2* reverses the direction and only allocates every other top tracks. Finally, the *Phase 3* reverses the direction again and allocates those residual top tracks. Although Z-alloc can effectively delay the timing of allocating top tracks, *the possibility of storing frequently-updated data onto bottom tracks remains.* To relocate those frequently-updated data, data relocation, such as the block swap [13] and the data hotness aware movement [15], exchanges frequently-updated data on bottom tracks with infrequently-updated data on top tracks. In other words, update frequency tracking is required for hot/cold data separation. Nevertheless, *update frequency could change overtime according to locality and induce additional relocation overhead when the locality*

²Deduplicated data chunks refer to data chunks that have undergone the data deduplication process.

shifts. Lastly, an example of track buffering is the top buffering [13], which reserves a number of top tracks for buffering bottom track updates to delay or avoid the track swap. Even though previous strategies can effectively reduce the number of track rewrites on IMR drives, *the possibility of further reducing the amount of written data or facilitating IMR data management through data deduplication has received much less attention.*

B. Data Deduplication

Since 2000s, data deduplication techniques have been commercialized and widely adopted by cloud storage providers. For instance, Microsoft and EMC report that roughly 50% and 85% of data in their storage systems are duplicated [16]. The process of data deduplication can be summarized as Figure 3. First, data duplication are identified at either *block-* or *chunk-level*. For chunk-level deduplication, incoming data streams are divided into smaller data chunks via either fixed-size chunking (FSC) or content-defined chunking (CDC), where CDC can achieve higher deduplication ratio than FSC. Second, each data chunk is labeled with a fingerprint generated by hash functions, such as MD5 and SHA-256, and are compared with other fingerprints to check whether the data chunk is duplicate or not. The process of comparison is known as *fingerprint indexing*, which can be accelerated by Bloom filters [16].

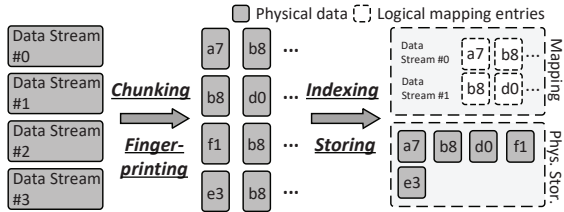


Fig. 3. The process of chunk-level deduplication, in which data streams go through the chunking, fingerprinting, indexing, and storing steps for data deduplication.

Finally, mapping entries are maintained between the stored physical copy of the data chunk and the logically deduplicated data chunks or files. In this study, data deduplication mapping entries are assumed to be maintained within the ITL after the host system conducts deduplication and passes the results through the reserved field in the ATA command set. This paper reveals that the reference count can be repurposed to indicate whether a data chunk will be updated. Notably, rather than using the reference count for access frequency prediction, data with a reference count greater than 1 remains static and receives no updates unless the reference count drops to 1. According to the investigation of this study, *such an indication from data deduplication techniques has not been explored to facilitate the data management of IMR drives.*

C. Motivation

This section provides an illustrative example to explain the existing inefficiency of the state-of-the-art space allocation and data relocation strategies, namely Z-alloc and block swap, on IMR drives. As shown in Figure 2(b), existing Z-alloc divides the track allocation into 3 phases based on the ratio of space utilization. However, as shown in Figure 4(a), in *Phase 1*, all data, including frequently-updated data, can only be written to bottom tracks. If subsequent updates to the frequently-updated data happen during either *Phase 2* or *Phase 3*, track rewrites

or block swaps become inevitable and induce performance overhead, as shown in Figure 4(b). Such an unwanted overhead can be avoided if data that will not be updated, namely deduped data chunks with reference count larger than 1, can be identified and proactively stored on bottom tracks. Consequently, the adjacent top tracks of bottom tracks containing deduplicated data chunks can be allocated for data that could be updated directly in *Phase 1*.

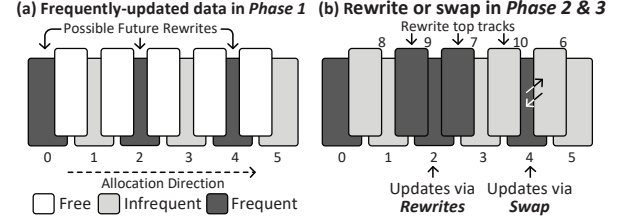


Fig. 4. Motivational example, in which frequently-updated data are written on bottom tracks in the Phase 1 of Z-alloc and induce track rewrites or swaps in Phase 2 and 3.

To relocate frequently- (*i.e.*, hot) and infrequently-updated (*i.e.*, cold) data, the existing block swap strategy needs to attain the hot and cold data information on the block basis. Typically, such hot/cold information is obtained from keeping the access count of each data block. Other strategies, such as Least Recently Used (LRU), can be also utilized; however, both the access count and LRU mechanism could be affected by temporal locality, leading to frequent block swaps under thrashing scenarios. The reference count in data deduplication techniques, on the other hand, provides a clear indication of whether a stored data chunk may be updated—specifically, stored data chunks with a reference count greater than 1 are static and receive no updates unless the reference count drops to 1. It would be beneficial if deduplication information could substitute for access count or LRU mechanisms, as this approach would reduce the influence of temporal locality while identifying blocks that remain static.

III. DEDUPPLICATION-ASSISTED DATA RELOCATION (DADR) SCHEME

A. Overview

To facilitate the data management of IMR drives, this paper presents a deduplication-assisted data relocation (DADR) scheme with the goal of lowering the number of track rewrites by leveraging the data deduplication information. Figure 5 illustrates the system architecture, showing how deduplication metadata facilitates the DADR scheme in managing IMR drives. In the considered system architecture, the generation of deduplication fingerprint is conducted at the host system for the better processing capability. Then, deduplication index, the logical address, and the original data are passed through conventional write requests of ATA command. Under this implementation, a two-level mapping table is presented to keep track of both the L2P mapping entries of the original ITL and the data deduplication information. Logical addresses of stored data chunks are mapped to deduped entries based on the entry indices, and each deduped entry records the fingerprint, the reference count, and the physical address. As deduplication information is shared between the host system and the proposed scheme, deduped entries are cached onto the host system and periodically flush are conducted to ensure consistency. Alternatively, if a specialized

controllers or special intellectual property (IP) cores can be integrated into the IMR drives for fingerprint generation, the fingerprint could be generated within the ITL, eliminating the need to pass it down from the host system.

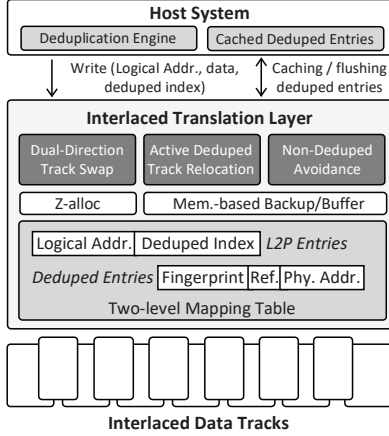


Fig. 5. The system architecture of DADR scheme, in which Z-alloc is utilized with the aids of proposed components.

Based on the two-level mapping table, the main components of the DADR scheme are illustrated below. First, the *dual-direction track swap* is proposed to swap to-be-updated bottom tracks with top tracks that have the largest deduped reference count. The main rationale behind the dedupe-based track swap is that the only physical copy of deduped data chunks remains static if it is referenced by more than one logical data chunk. In other words, this physical copy remains unchanged even when one of its logical data chunk is being updated, which either remaps to another deduped physical data chunk or triggers copy-on-write (COW) to create another new physical data chunk. Second, based on the state-of-the-art allocation method (i.e., Z-alloc), the *active deduped track relocation* is included to actively relocate non-deduped and deduped data to the top and bottom tracks, respectively, during the *Phase 1* of IMR track allocation. Lastly, the *non-deduped avoidance* is included during *Phases 2* and *3* to skip allocation of those top tracks with their adjacent bottom tracks storing non-deduped data chunks (i.e., reference count equals to 1), and introduces *Phase 4* to utilize those residual top tracks after *Phase 3*. In summary, this paper presents a pioneering design that first integrates data deduplication on IMR drives and leverage deduplication information to facilitate data management.

B. Dual-Direction Track Swap

To avoid frequent track rewrites, previous studies [13, 15] have suggested that swapping hot bottom tracks with cold top tracks is an effective action after valid top tracks start overlapping valid bottom tracks. While most of the previous designs rely on the update frequency of each data track for identify hot/cold data, the proposed DADR scheme includes the dual-direction track swap to utilize the reference count of data deduplication mechanism for track swaps. Accordingly, the to-be-updated bottom tracks with the reference count of 1 are going to be swapped with the top track that has the highest reference count. In an extreme case where all top tracks have a reference count of 1, the RMW operation will be triggered to update the bottom tracks and preserve the overlapped valid top tracks. Nevertheless, according to our experimental results,

such an extreme case has not been observed with real-world workloads. An example is given in Figure 6 to better illustrate the above procedure. In this example, an update request is made to *Track 2*, and the overlapping *Track 9* and *Track 7* are buffered to prevent data loss. Data on *Track 2* are swapped with the top track with the highest reference count, which is the *Track 9*. Subsequently, the original data on *Track 9* are relocated to *Track 9* and the original data on *Track 2* are merged with updates and stored to *Track 9*.

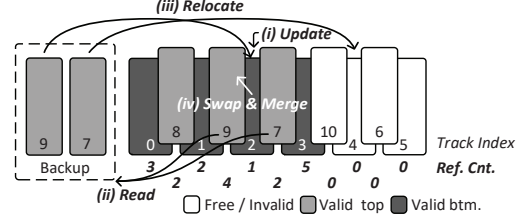


Fig. 6. Dual-direction track swap, which lowers the possibility of future rewrites by swapping and relocating tracks based on the deduped reference count.

Notably, in addition to proactively swapping non-deduped bottom and deduped top tracks, the proposed dual-direction track swap further leverages the deduped reference count to relocate adjacent deduped top tracks of the to-be-updated bottom tracks when the to-be-updated bottom track is swapped with a deduped top tracks. This is because, as shown in Figure 6, overlapping top tracks of to-be-updated bottom tracks are buffered. If those top tracks contains deduped data chunks, it would be beneficial to relocate the buffered top track to an invalid or a non-deduped bottom track since deduped data chunks (i.e., the reference count is larger than 1) remains static. Such an approach allows top tracks to be reserved for data that could possibility be updated. In Figure 6, during the swap of *Track 2* and *Track 9*, the *Track 7* with a reference count of 2 is also read out. To fully utilize the track rewrite opportunity and avoid future rewrite overhead, the *Track 7* with the reference count of 2 is moved to a free bottom track, namely *Track 4*, during the updates of *Track 2*. Then, *Track 7* becomes free again for future allocation. Such an approach makes the rewrite overhead worthwhile and fully utilizes the track rewrite opportunity.

C. Active Deduped Track Relocation

Following the Z-alloc method on IMR drives, *Phase 1* of Z-alloc could force frequently-updated data to be stored on bottom tracks and leads to track rewrites or block swaps during *Phase 2* and *Phase 3*. In other words, it would be beneficial if infrequently- and frequently-updated data can be redirected to bottom and top tracks during *Phase 1* of Z-alloc. In the proposed DADR scheme, the active deduped track relocation, as summarized in Figure 7, is included to relocate data according to their deduped reference count, which can be observed while data are buffered on memory before being stored permanently. Since buffering is widely used on HDDs to improve response time, the active deduped track relocation does not alter the existing data flow. Data with a deduped reference count greater than 1 are stored on bottom tracks, while non-deduped data with a reference count of 1 are proactively stored on the top tracks above them. Top tracks that are on top of bottom tracks containing deduped data chunks are referred to as allocatable top tracks. If allocatable top tracks do not exist, those non-deduped

data are still stored on bottom tracks since writing to top tracks on top of free bottom tracks leads to RMW operations while those free bottom tracks are allocated in the future. Notably, since deduped data chunks are typically smaller than a track of IMR drives, multiple data chunks are stored on the same tracks, which are non-deduped or deduped tracks. That is, data chunks are stored onto data tracks based on whether their reference count is larger than one or not. This is because the proposed scheme is designed based on whether a data block will be updated or not, and the ranking of the reference counts makes no difference.

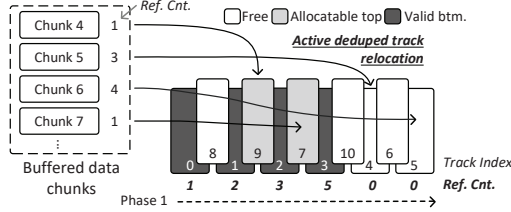


Fig. 7. Active deduped track relocation, which aims to avoid storing non-deduped data chunks onto bottom tracks in *Phase 1* of Z-alloc.

D. Non-Deduped Avoidance

To further leverage deduplication information to lower track rewrite overhead on IMR drives, the proposed DADR scheme introduces the non-deduped avoidance into the *Phase 2* and *Phase 3* of Z-alloc to delay the allocation of top tracks that are on top of bottom tracks containing non-deduped data chunks. Meanwhile, the allocation of those top tracks is delayed until a newly-included *Phase 4* after the *Phase 3* finishes. Such approach further reduces the possibility of track rewrites since those bottom tracks with non-deduped data chunks can still be freely updated without inducing track rewrites during *Phase 2* and *Phase 3*. Notably, in this study, instead of completely replacing the state-of-the-art space allocation method on IMR drives, the proposed DADR scheme explores the merits of deduplication information to assist the data management of IMR drives and reduce the possibility of track rewrites. In other words, the components of the proposed DADR scheme can be integrated to other allocation methods on IMR drives, while integrating data deduplication to reduce the size of written data.

IV. PERFORMANCE EVALUATION

A. Experiment Setup

In this session, experiments are conducted to evaluate the effectiveness of the proposed DADR scheme in terms of reducing the accumulated read and write latencies, as well as the number of rewritten data tracks. Since the current implementation involves altering the ITL of IMR drives, both the existing Tracklace [13] approach and the proposed DADR scheme are implemented and simulated on an IMR simulator [6]. Notably, the Tracklace includes methodologies such as Z-alloc, block swap, and top buffer approaches. The evaluation uses three types of workloads: two real-world traces from the FSL-Dedup Traces dataset and one synthetic trace with a lower data deduplication ratio. The characteristics of each trace used in the experiment are detailed in Table I.

The two real-world traces, labeled *user000* and *user002*, provide valuable insights into the performance of storage systems under deduplication conditions. In the selection process for

TABLE I
WORKLOAD CHARACTERISTICS.

Traces	Write Size (GB)	Dedupe (%)	Update size (GB)
user000	70.8	16.9	44.1
user002	210.9	27.2	128
synthetic	130.8	14.4	98.69

these two traces, the main consideration is the peak data chunk sizes and the aim to understand how data chunk size impacts the effectiveness of the proposed scheme. The peak chunk sizes for the two traces are near 0 bytes and approximately 250,000 bytes. Given that FSL-Dedup traces capture a snapshot of a deduplication-enabled storage system, additional data update requests were manually and randomly generated during this evaluation to simulate real-world usage. The synthetic trace, on the other hand, was generated with a normal distribution of data write sizes for each entry, using a mean of 128 KB and a standard deviation of 30 KB. This synthetic trace helps to model a scenario with different deduplication ratios for helping to understand the effectiveness and performance trend of the proposed scheme, thus providing a contrast to real-world traces.

B. Experimental Results

To eliminate the effect of data deduplication and focus on investigating the effectiveness of data management of the DADR scheme, the improvement percentage shown in this paper is derived by comparing the proposed DADR scheme and the Tracklace with data deduplication enabled (*i.e.*, Tracklace Dedupe). The experimental results of Tracklace without enabling data deduplication are also included for completeness. In addition, to understand the capability of the proposed DADR scheme under different space utilization scenarios, three ratios of space utilization are chosen, which are 50%, 80%, and 90%. Accordingly, the accumulated latency comparisons are first shown in Figure 8. When the ratio of space utilization is 50%, the accumulated latency can be reduced by 59.96% on average when compared with Tracklace Dedupe. This is because the DADR scheme can actively relocate data to the bottom and top tracks based on the reference count of the deduplication mechanism to differentiate static data or data that could be possibly updated. Meanwhile, the reason why Tracklace performs worse than Tracklace Dedupe is that Tracklace already enters the *Phase 2* of Z-alloc, the top buffer of Tracklace is enforced for every update to bottom tracks, thus inducing additional overhead.

After the ratio of space utilization grows up to 80% and 90%, the proposed scheme achieves latency reductions of 26.68% and 26.06% on average, respectively. The reason for the reduction drop is that the dual-direction block swap of the proposed scheme is triggered to relocate data due to updates to bottom tracks. Nonetheless, the reduction is still positive and up to 32% for *user002* at 90% space utilization. On the other hand, as one of the design goals of the DADR scheme is to reduce track rewrites, the accumulated rewrite data size is reported in Figure 9. For 50% space utilization, the DADR scheme induces no track rewrites since the dual-direction block swap is not triggered. On the contrary, Tracklace requires RMW operations when flushing the top buffer to the bottom tracks. After the ratio of space utilization grows to 80% and 90%, the average reductions are 4.93% and 19.63%, respectively. The reason is

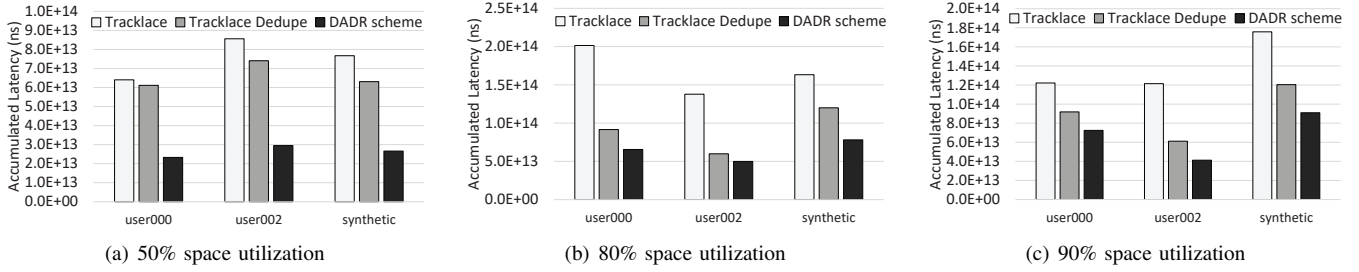


Fig. 8. Comparison of accumulated latency under different space utilization, including the latency of data reads/writes for serving incoming requests and the latency of data rereads/rewrites during block swap for all three methods and top buffer flush, which is enabled for Tracklace and Tracklace Dedupe.

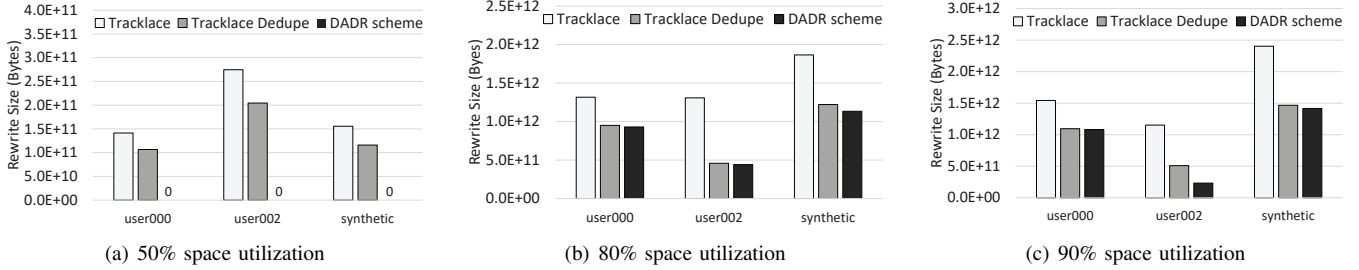


Fig. 9. Comparison of accumulated rewrite size of block swaps under different space utilization. This comparison is included to show the number of track rewrites between each compared methods.

that, at 80% space utilization, the dual-direction block swap is constantly triggered. Then, at 90%, there are much fewer block swaps as most of the active updated data are relocated to top tracks. In other word, even when the space utilization is at 90%, the included active deduped track relocation is still effective in terms of reducing the amount of data rewrite.

V. CONCLUSION

To avoid the locality shifting issue of conventional data management on IMR drives, this study introduces a pioneering approach called the deduplication-assisted data relocation (DADR) scheme. The DADR scheme leverages data deduplication information to optimize data management on IMR drives and strategically relocates deduped data to the bottom tracks while moving non-deduped data with a higher likelihood of updates to the top tracks. Notably, instead of tracking updating frequency or trying to reflect updating frequency through the deduplication information, the proposed DADR scheme is designed based on the fact that deduped data remain static unless the reference count drop to 1. Accordingly, the DADR scheme actively redirects data to the bottom and top tracks and delays the allocation of top tracks on non-deduplicated bottom tracks. With the above mechanism, the DADR scheme demonstrates a significant reduction in both track rewrites and accumulated read/write latency. On average, it achieves a 41.34% reduction in track rewrites and a 37.75% reduction in accumulated read/write latency compared to the existing Tracklace approach with data deduplication enabled. By minimizing track rewrites, the DADR scheme presents a promising approach for managing IMR drives data deduplication.

REFERENCES

- [1] A. Amer, J. Holliday, D. D. E. Long, E. L. Miller, J.-F. Paris, and T. Schwarz, "Data management and layout for shingled magnetic recording," *IEEE Transactions on Magnetics*, vol. 47, no. 10, pp. 3691–3697, 2011.
- [2] S. Greaves, Y. Kanai, and H. Muraoka, "Shingled recording for 2–3 tbit/in²," *IEEE Transactions on Magnetics*, vol. 45, no. 10, pp. 3823–3829, 2009.
- [3] W. He and D. H. Du, "SMaRT: An approach to shingled magnetic recording translation," in *15th USENIX Conference on File and Storage Technologies (FAST 17)*, Santa Clara, CA, Feb. 2017, pp. 121–134.
- [4] A. Aghayev and P. Desnoyers, "Skylight—A window on shingled disk operation," in *13th USENIX Conference on File and Storage Technologies (FAST 15)*, Santa Clara, CA, Feb. 2015, pp. 135–149.
- [5] E. Hwang, J. Park, R. Rauschmayer, and B. Wilson, "Interlaced magnetic recording," *IEEE Transactions on Magnetics*, vol. 53, no. 4, pp. 1–7, 2017.
- [6] S.-H. Chen and K.-H. Huang, "Leveraging journaling file system for prompt secure deletion on interlaced recording drives," *IEEE Transactions on Emerging Topics in Computing*, vol. 11, no. 3, pp. 619–634, 2023.
- [7] S. Yoon and E. Hwang, "Joint multi-level readback for array-reader-based interlaced magnetic recording," *IEEE Transactions on Magnetics*, vol. 57, no. 3, pp. 1–5, 2021.
- [8] W. Digital, "Product brief: Wd red nas hdd - western digital," accessed on 10.22.2023. [Online]. Available: https://documents.westerndigital.com/content/dam/doc-library/en_us/assets/public/western-digital/product/internal-drives/wd-red-hdd/product-brief-western-digital-wd-red-hdd.pdf
- [9] Y.-J. Chuang, S.-H. Chen, Y.-H. Chang, Y.-P. Liang, H.-W. Wei, and W.-K. Shih, "Dstl: A demand-based shingled translation layer for enabling adaptive address mapping on smr drives," *ACM Trans. Embed. Comput. Syst.*, vol. 19, no. 4, jul 2020.
- [10] S. Granz, W. Zhu, E. C. S. Seng, U. H. Kan, C. Rea, G. Ju, J.-U. Thiele, T. Rausch, and E. C. Gage, "Heat-assisted interlaced magnetic recording," *IEEE Transactions on Magnetics*, vol. 54, no. 2, pp. 1–4, 2018.
- [11] Y. Liang, M.-C. Yang, and S.-H. Chen, "Magic: Making imr-based hdd perform like cmr-based hdd," *IEEE Transactions on Computers*, vol. 71, no. 3, pp. 643–657, 2022.
- [12] K. Gao, W. Zhu, and E. Gage, "Write management for interlaced magnetic recording devices," Patent US9 508 362B2, Nov. 2016.
- [13] F. Wu, B. Li, B. Zhang, Z. Cao, J. Diehl, H. Wen, and D. H. Du, "Tracklace: Data management for interlaced magnetic recording," *IEEE Transactions on Computers*, vol. 70, no. 3, pp. 347–358, 2021.
- [14] M. H. Hajkazemi, A. N. Kulkarni, P. Desnoyers, and T. R. Feldman, "Track-based translation layers for interlaced magnetic recording," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, Renton, WA, Jul. 2019, pp. 821–832.
- [15] Y. Liang and M.-C. Yang, "Move-on-modify: An efficient yet crash-consistent update strategy for interlaced magnetic recording," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 97–102.
- [16] W. Xia, H. Jiang, D. Feng, F. Dougliis, P. Shilane, Y. Hua, M. Fu, Y. Zhang, and Y. Zhou, "A comprehensive study of the past, present, and future of data deduplication," *Proceedings of the IEEE*, vol. 104, no. 9, pp. 1681–1710, 2016.
- [17] E. Shriver, C. Small, and K. A. Smith, "Why does file system prefetching work?" in *Proceedings of the Annual Conference on USENIX Annual Technical Conference*, ser. ATEC '99, USA, 1999, p. 6.
- [18] J. Park, J. Kim, Y. Kim, S. Lee, and O. Mutlu, "DeepSketch: A new machine Learning-Based reference search technique for Post-Deduplication delta compression," in *20th USENIX Conference on File and Storage Technologies (FAST 22)*, Santa Clara, CA, Feb. 2022, pp. 247–264.
- [19] S. Tarun, R. S. Batth, and S. Kaur, "A scheme for data deduplication using advance machine learning architecture in distributed systems," in *2021 International Conference on Computing Sciences (ICCS)*, 2021, pp. 53–60.
- [20] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed Solid-State drive," in *6th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 14)*, Philadelphia, PA, Jun. 2014.
- [21] S.-H. Chen, C.-F. Wu, M.-C. Yang, and Y.-H. Chang, "A file-oriented fast secure deletion strategy for shingled magnetic recording drives," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 8, pp. 2463–2476, 2022.